Bharat Setu — भारत सेतु

Comprehensive Project Report

Bridging the Digital Divide with Agentic Governance

Project Name: Bharat Setu (भारत सेतु — "India's Bridge") **Version:** 2.0.0 **Platform:** Web (PWA) + Android (Capacitor) **Primary Language:** Hindi (with 22 Indian language support) **Target Users:** 1.3 Billion Indian Citizens + Government Officials

---

Table of Contents

1. Executive Summary
2. Problem Statement
3. Solution Overview
4. Detailed Feature Analysis
5. Technical Architecture
6. Tech Stack Details
7. AI/ML Pipeline — Deep Dive
8. API Endpoints Reference
9. State Management & Data Model
10. Internationalization (i18n)
11. Security & Content Safety
12. Deployment Architecture
13. File-Level Project Map
14. Conclusion

---

1. Executive Summary

Bharat Setu is a production-grade, AI-powered digital governance platform built with Next.js 14 and powered by Microsoft Azure AI services. The platform is designed to democratize access to Indian government services for citizens who may not be digitally literate, may speak only regional languages, or may not know which department handles their problem.

The platform deploys a **"Council of Five"** — five specialized AI agents, each an expert in a specific governance domain (civic services, health, government schemes, finance, and legal aid). When a citizen describes their problem in any of 22 Indian languages — by typing or speaking — the system automatically classifies the query, routes it to the correct specialist agent, and generates a contextual, actionable response in the citizen's own language.

Beyond the AI chat interface, Bharat Setu includes a complete **SOS emergency system** with multi-channel responder dispatch, a **grievance filing workflow** with photo evidence and Azure Computer Vision analysis, a **government scheme matcher** backed by Azure AI Search, a **DIGIPIN-based location system** using ISRO's geocoding standard, and a **civic gamification engine** that rewards citizens for engagement.

The platform operates as a **Progressive Web App (PWA)** with offline support, can be compiled into a native **Android app** via Capacitor, and is containerized with **Docker** for cloud deployment.

---

2. Problem Statement

India's digital governance ecosystem presents several critical challenges for its 1.3 billion citizens:

2.1 Fragmented Government Portals

Indian government services are spread across hundreds of portals — central, state, and local — each with different interfaces, languages, and procedures. A citizen trying to apply for a ration card, track a PM-KISAN payment, or file a police complaint must navigate completely different systems. **Bharat Setu unifies all these touchpoints into a single conversational interface.**

2.2 Language Barriers

While India has 22 officially recognized languages and hundreds of dialects, most government portals operate primarily in English or Hindi. A Tamil-speaking farmer in Tamil Nadu or a Bengali-speaking worker in West Bengal faces significant barriers when trying to access digital services. **Bharat Setu supports all 22 scheduled languages of India**, with AI-generated responses that naturally mix the user's language with relevant English technical terms (scheme names, portal URLs).

2.3 Low Digital Literacy

According to government data, a significant portion of India's rural population has limited digital literacy. Expecting these users to fill online forms, navigate dropdown menus, and understand bureaucratic terminology is unrealistic. **Bharat Setu uses a voice-first approach** — citizens can simply speak their problem and receive guidance in their language.

2.4 No Unified Emergency Response

India has multiple emergency numbers (100 for police, 108 for ambulance, 101 for fire, 1091 for women's helpline, 1098 for child helpline), but no unified platform that can simultaneously alert all relevant responders based on the nature of the emergency. **Bharat Setu's SOS engine dispatches alerts to multiple channels simultaneously**, with intelligent context classification that activates specialized responders (women's safety, child protection, cyber crime) based on keywords.

2.5 Citizen-Government Trust Deficit

Citizens often feel disconnected from the resolution of their complaints. They file grievances but never hear back. **Bharat Setu introduces transparency through trackable cases, department trust scores, and a civic gamification system** that makes citizen participation rewarding.

---

3. Solution Overview

Bharat Setu addresses each of these problems through a layered architecture:

Layer 1 — Conversational AI Interface: Citizens interact through text or voice in their preferred language. The AI understands intent, classifies the domain, and routes to the appropriate specialist agent.

Layer 2 — Specialist Agent Responses: Each of the 5 agents has deep domain knowledge embedded in their system prompts, enriched with the citizen's profile (Aadhaar-linked), DIGIPIN location, and conversation history for personalized guidance.

Layer 3 — Actionable Workflows: Beyond conversation, the platform drives tangible actions — filing grievances with photo evidence, matching citizens to eligible government schemes, triggering SOS emergencies, tracking case status, and generating pre-filled government forms.

Layer 4 — Civic Engagement Loop: A karma-based gamification system rewards citizens for filing grievances, tracking issues, joining collective action clusters, and engaging with the platform. Higher karma unlocks additional permissions and community leadership roles.

Layer 5 — Government Dashboard: A separate interface for government officials provides case management, analytics, alerts, and administrative tools to monitor and respond to citizen needs.

---

4. Detailed Feature Analysis

4.1 Multi-Agent AI System — "Council of Five"

The heart of Bharat Setu is its multi-agent architecture, inspired by Microsoft's AutoGen 0.4 pattern. Five specialized AI agents handle different governance domains:

Nagarik Mitra (नागरिक मित्र) — Civic Services Agent Handles all municipal and civic complaints. This agent assists with birth/death certificates, property registration, DIGIPIN-based address verification, municipal services, RTI applications, and government form assistance. It understands complaints about roads, water supply, electricity, garbage, drainage, street lights, and local government operations. When a citizen says "मेरे मोहल्ले में सड़क टूटी है" (the road in my colony is broken), this agent generates actionable guidance — which department to contact, what reference numbers to mention, and what timelines to expect.

Swasthya Sahayak (स्वास्थ्य सहायक) — Health Assistant Agent Handles all health and wellness queries. This agent assists with Ayushman Bharat scheme enrollment, nearest health facility identification using DIGIPIN, vaccination schedules, health records, and emergency health guidance. It always recommends calling 108 (ambulance) or 112 (national emergency) for genuine emergencies. The agent is trained to recognize symptoms described colloquially in regional languages — for example, "पेट अजीब लग रहा है" (stomach feels strange) correctly routes to this agent via keyword detection.

Yojana Saathi (योजना साथी) — Welfare Schemes Agent Handles all government scheme queries. This is perhaps the most impactful agent — it helps citizens navigate India's vast ecosystem of 800+ government welfare schemes. It assists with PM-KISAN payments, MGNREGA job cards, PM Awas Yojana housing, PDS ration cards, pensions, scholarships, Mudra loans, crop insurance, LPG subsidies, and more. The agent uses the citizen's profile data (income, occupation, BPL status, state, caste category) to proactively suggest schemes they may be eligible for.

Arthik Salahkar (आर्थिक सलाहकार) — Finance Advisory Agent Handles banking, financial literacy, and fraud-related queries. This agent assists with Jan Dhan account services, Mudra loan applications, tax filing (simplified for rural users), UPI payment issues, savings guidance, and — critically — helps victims of financial fraud (OTP scams, phishing, unauthorized UPI transactions). The agent detects fraud-related keywords in multiple languages and provides step-by-step guidance on reporting to cyber crime cells.

Vidhi Sahayak (विधि सहायक) — Legal Aid Agent Handles legal rights and FIR/police-related queries. This agent assists with NALSA free legal aid eligibility, FIR filing guidance, police complaint procedures, land and property dispute resolution, eviction issues, bail and arrest information, consumer court procedures, RTI applications, and understanding legal documents in simple language. It is specially trained to recognize domestic violence, harassment, and women's safety keywords, ensuring victims receive immediate, appropriate guidance.

How Agent Routing Works — The 7-Layer Pipeline

The routing pipeline is a carefully designed cascade of classifiers, each optimized for different scenarios:

Layer 1 — Raw Script Detection: Before any translation, the system runs regex pattern matching against the original message in Devanagari/regional scripts. This catches high-confidence Hindi keywords like "बैंक"

(bank), "डॉक्टर" (doctor), "योजना" (scheme), "पुलिस" (police), "सड़क" (road) without requiring a translation API call. This layer is the fastest (sub-millisecond) and handles the most common queries.

Layer 2 — Azure Translator: The message is translated to English using Azure Cognitive Services Translator API. The translation is cached in an in-memory Map to avoid repeated API calls for identical texts. If Azure Translator is unavailable, the system falls back to MyMemory (free translation API).

Layer 3 — English Keyword Overrides: After translation, high-confidence English patterns are matched — "ration card" → yojana_saathi, "domestic violence" → vidhi_sahayak, "feeling sick" → swasthya_sahayak. This catches unambiguous cases without waiting for the AI model.

Layer 4 — Phi-4 Mini Classification: Microsoft's Phi-4 model (accessed via GitHub Models free tier) performs zero-shot intent classification. The model receives a carefully crafted system prompt that defines each agent's scope and few-shot examples. A 5-second race timeout ensures that if the model is cold-starting, the system doesn't block.

Layer 5 — Local TF-IDF Fallback: If Phi-4 doesn't respond within 5 seconds, a local Python TF-IDF + SVC classifier (running on port 5001) provides a fast fallback classification.

Layer 6 — GPT-4o Background Verification: While GPT-4o-mini generates the response, a background routing check runs in parallel using few-shot examples. If it disagrees with Phi-4's classification, the higher-quality model's verdict takes precedence.

Layer 7 — Client-Side Detection: As a last resort, client-side keyword detection in the browser provides a fallback classification.

4.2 Voice Assistant

The voice assistant is built on **Azure Speech Services** and provides:

Speech-to-Text (STT): The browser captures audio using one of two strategies — OGG/Opus encoding via MediaRecorder (preferred on supporting browsers) or raw WAV encoding via AudioContext + ScriptProcessor (universal fallback). The captured audio is sent to the `/api/stt` endpoint, which forwards it to Azure Speech for transcription. The system supports all 22 Indian languages for recognition.

Text-to-Speech (TTS): Agent responses are converted to speech via Azure Speech service, allowing the platform to "speak back" to the user in their preferred language. This is crucial for users with limited reading ability.

Intent Classification via Voice: Voice input goes through the same 7-layer routing pipeline as text input, ensuring consistent agent selection regardless of input mode.

Audio Processing Details: The `web-stt.ts` module implements a complete audio pipeline — microphone capture with echo cancellation, noise suppression, and auto-gain control; WAV encoding with RIFF header construction; OGG/Opus encoding via MediaRecorder; configurable maximum recording duration (default 7 seconds); and graceful cleanup of audio contexts and media streams.

4.3 Grievance Filing System

The grievance system is a complete civic complaint workflow:

Step 1 — Description: Citizens describe their complaint in any supported language. The description is checked against Azure Content Safety API to flag inappropriate content before processing.

Step 2 — Photo Evidence: Citizens can attach a photo of the issue (pothole, broken streetlight, overflowing garbage). The image is analyzed by **Azure Computer Vision (v3.2 API)** which extracts captions, tags, and

detected objects. For example, a photo of a broken road might produce: Caption: "A damaged road with visible potholes near residential area", Tags: ["road", "pothole", "damage", "infrastructure", "urban"], Objects: ["pothole", "road surface"].

Step 3 — AI Ticket Routing: The vision analysis, complaint description, and category are sent to **Phi-4** which generates structured ticket routing — the correct government department (e.g., "Municipal Corporation - Electrical Wing"), ward assignment, priority level (HIGH/MEDIUM), and estimated resolution timeline. This replaces manual triage with AI-powered automatic routing.

Step 4 — DIGIPIN Location: The citizen's GPS coordinates are encoded into a DIGIPIN for precise location reference. This allows the system to identify which ward, zone, and jurisdiction the complaint falls under.

Step 5 — Collective Action: After submission, the system checks if other citizens in the same DIGIPIN zone have filed similar complaints. If a cluster exists (e.g., 14 people reporting water supply issues in zone 3829), the citizen is invited to join the collective action cluster for amplified impact.

4.4 DIGIPIN Location System

DIGIPIN (Digital Postal Index Number) is Bharat Setu's implementation of the **ISRO/India Post geocode specification**:

Encoding Algorithm: GPS coordinates (latitude, longitude) are normalized within India's geographic bounds (lat 6.5°–37.6°, lng 68.1°–97.4°), converted to 20-bit fixed-point values, interleaved bit-by-bit, and mapped through a 20-character set (`2 3 4 5 6 7 8 9 C F H J M P Q R V W X Y`) specifically chosen to avoid visually ambiguous characters (no O/I/L/B/A).

Format: The resulting 10-character code is formatted as `XXXX–XXX–XXX` with a hierarchical structure — the first 4 characters identify the zone, the next 3 identify the block, and the last 3 identify the cell. This provides progressively finer location resolution.

Usage Throughout Platform: DIGIPIN is used for grievance location tagging, SOS emergency dispatch, scheme eligibility (state-level filtering), nearest facility identification, and collective action zone clustering.

4.5 SOS Emergency System

The SOS engine is one of the most complex subsystems in Bharat Setu, designed for life-critical situations:

Trigger Mechanisms:

- **Hold-to-activate button** — Press and hold the SOS button for a configurable duration to prevent accidental triggers
- **Voice trigger** — Optional voice-activated SOS for hands-free emergency situations
- **Hardware trigger** — The SOSHardwareTrigger component listens for device button events

GPS Capture: The system requests high-accuracy GPS with `enableHighAccuracy: true`, an 8-second timeout, and a 5-second maximum age for cached positions. The coordinates are immediately encoded into a DIGIPIN.

Context Classification: The system analyzes the emergency description (if provided) and user profile to determine which specialist channels to activate:

- **Women's safety** — Detected via keywords (assault, harassment, domestic violence, etc.) in Hindi, English, and Tamil, or if the user's gender is female
- **Child safety** — Detected via keywords (child, minor, kidnap, trafficking) or if the user's age is under 18
- **Cyber crime** — Detected via keywords (fraud, scam, hacking, OTP, phishing)

- **Disaster** — Detected via keywords (flood, fire, earthquake, cyclone, building collapse)

Responder Fan-Out: Alerts are dispatched simultaneously to all relevant responders using `Promise.allSettled()`. Each responder receives a customized SMS/message with the citizen's name, mobile number, DIGIPIN, GPS coordinates, and a Bing Maps link. The base responders (Police 100, Ambulance 108, Fire 101, Legal Aid 15100) are always notified. Conditional responders (Women's Helpline 1091, CHILDLINE 1098, Cyber Crime 1930, NDRF 1078) are added based on context classification.

Offline Fallback: When internet is unavailable, the system generates an `sms:` deep-link that opens the native SMS app pre-populated with the emergency message and location, allowing the citizen to send an SOS via SMS without internet connectivity.

SMS Integration: The platform supports two Indian SMS providers — **Fast2SMS** and **MSG91** — for programmatic SMS dispatch. For voice calls, **Exotel** integration is available (optional/paid).

Webhook Support: Configurable webhook URLs can be set up for Police, Mahila (Women's Safety), CHILDLINE, NDRF, Cyber Crime, and Legal Aid departments, enabling direct integration with government alert systems.

4.6 Government Scheme Scanner

The scheme scanner helps citizens discover and apply for government welfare schemes:

Azure AI Search Backend: The platform maintains a cognitive search index (`schemes-index`) populated with 800+ Indian government schemes. The index is set up via the `setup-search-index.js` script (19KB) which defines the schema and uploads scheme data.

Search Capabilities: The API supports full-text search with OData filters for category, state, and income level. Selected fields include scheme name, ministry, description, eligibility criteria, benefits, application URL, deadline, and match score.

Profile-Based Matching: When a citizen's profile is available (income, occupation, state, BPL status), the system can filter schemes based on eligibility criteria, ensuring relevant recommendations.

Demo Fallback: When Azure AI Search is unavailable, the system returns curated demo data for major schemes (PM-KISAN, Ayushman Bharat PM-JAY, PM Awas Yojana, MGNREGA, PM Mudra Yojana) ensuring the feature remains functional.

4.7 Civic Gamification — Karma System

Bharat Setu implements a civic gamification layer to encourage citizen participation:

Karma Points:

- Filing a grievance: +25 karma
- Creating a tracked case: +50 karma
- Joining a collective action cluster: +5 karma

Role Progression:

Karma Range	Role	Permissions
0–50	Citizen	Basic actions only
51–149	Contributor	Highlight issues, community engagement

150+	Community Head	Cluster issues, priority flagging, community engagement

Redeemable Rewards: Citizens can spend karma on rewards through the `redeemReward()` function in the store, creating a tangible incentive loop.

4.8 Intelligence Engine

The intelligence engine provides advanced analytics capabilities:

Collective Action Clusters: When multiple citizens in the same DIGIPIN zone file similar grievances, the system groups them into clusters. For example, if 14 people in zone 3829 report water supply issues, a cluster `CLU-WTR-001` is created. When the cluster reaches a threshold (e.g., 10+ participants), its status changes to "amplified," signaling urgency to government officials.

Department Trust Scores: Each government department receives a 0-10 trust score calculated from three weighted metrics — average resolution time (35% weight), complaint backlog (25% weight), and citizen satisfaction rating (40% weight). This creates public accountability and helps citizens set expectations.

Civic Digital Twin — Predictive Analytics:

- **Hotspot Mapping:** Identifies geographic areas with concentrated complaints (e.g., "Sector 3, Baharpur — 47 water supply complaints, Critical severity, Rising trend")
- **Trend Insights:** Tracks category-level trends (e.g., "Water complaints surged 34% this month — peak summer demand starting")
- **Predictions:** AI-generated predictions like "87% probability of water crisis in Ward 14 in the next 2 weeks" based on complaint patterns, seasonal data, and infrastructure age

4.9 Government Dashboard

A separate interface for government officials featuring:

- **GovDashboard** — Overview metrics and KPIs for citizen engagement and case resolution
- **GovCaseManagement** (17.6KB) — Manage, assign, and resolve citizen cases with status tracking
- **GovAnalytics** (16.5KB) — Data visualization with Recharts for complaint trends, resolution rates, and department performance
- **GovAlerts** (14.9KB) — Alert management and citizen broadcasting capabilities
- **GovAdmin** (14.4KB) — Administrative controls for user management and system configuration
- **GovStatusBar** (10.4KB) — Real-time status indicators and quick metrics
- **NagarPulse** (9.4KB) — Community pulse monitoring for emerging issues

4.10 Rich Chat Interface

The AgentChat component (86KB) implements a full-featured conversational interface:

- **Multi-agent conversations** with seamless handoff between specialists
- **Rich Chat Cards** — Structured response cards with action buttons, portal links, and scheme details
- **Image upload and analysis** — Photos are analyzed by Azure Vision and the analysis is injected as context into the conversation
- **Form generation** — Dynamic government forms can be generated from chat context with auto-filled citizen profile data
- **Shared MCP Context** — When a citizen is handed off between agents, the conversation history is shared so the new agent doesn't ask the citizen to repeat information
- **Citizen profile injection** — The agent's system prompt includes the citizen's Aadhaar-verified profile (name, age, income, location, enrolled schemes) for personalized responses

4.11 PWA & Mobile App

- **Progressive Web App** — Full PWA manifest with standalone display, portrait orientation, saffron (#FF9933) theme color
- **Service Worker** (4.4KB) — Caches critical assets for offline access
- **Capacitor** — Native Android app shell via @capacitor/android for distribution on Google Play Store
- **App Shortcuts** — SOS Emergency, Scheme Scanner, File Grievance accessible from the home screen long-press
- **Web Share Target** — Accepts shared images and audio from other apps for direct grievance filing

4.12 Onboarding & Citizen Profile

The Onboarding component (78KB) guides new users through:

- **Language selection** — Choose from 22 Indian languages
- **Profile creation** — Name (Hindi + English), DOB, gender, mobile, address, district, state, PIN code, DIGIPIN
- **Economic profile** — Occupation, annual income, BPL card status, ration card type (AAY/BPL/APL)
- **Aadhaar linking** — Simulated Aadhaar verification for identity authentication
- **Scheme mapping** — Auto-detection of currently enrolled and eligible government schemes based on profile data
- **Emergency contacts** — Add trusted contacts for SOS notification

---

5. Technical Architecture

The application follows a modern, layered architecture:

Presentation Layer (Client): Next.js App Router serves the React 18 application with TypeScript. The UI uses a hybrid approach — static stitched screens for government portal views and reactive React overlays for high-trust interactions (chat, voice, grievance, SOS). Tailwind CSS provides styling, Framer Motion handles animations, and Zustand manages global state.

API Layer (Server): Next.js Route Handlers serve as the backend, providing 12+ API endpoints. Each route handler communicates with external AI services (Azure, GitHub Models) and implements extensive fallback chains to ensure availability.

AI/ML Layer: Multiple AI models work in concert — Phi-4 Mini for classification, GPT-4o-mini for response generation, Azure Translator for language support, Azure Speech for voice, Azure Vision for image analysis, and Azure Content Safety for moderation.

External Services Layer: Azure Cognitive Services, GitHub Models (free tier), Fast2SMS/MSG91 for SMS, Exotel for voice calls, and configurable webhook endpoints for government integration.

---

6. Tech Stack Details

Frontend Technologies

Technology	Version	Purpose	Why Chosen
Next.js	14.2.x	Full-stack React framework	App Router, API routes, SSR/SSG, standalone builds for Docker

React	18.3.1	UI component library	Concurrent features, Suspense, hooks ecosystem
TypeScript	5.7.x	Type-safe JavaScript	Catches bugs at compile time, better DX
Tailwind CSS	3.4.x	Utility-first CSS framework	Rapid styling, consistent design, dark mode support
Zustand	5.0.0	State management	Minimal boilerplate, excellent TypeScript support, no context providers
Framer Motion	11.15.0	Animation library	Smooth page transitions, gesture support, layout animations
Recharts	2.15.0	Chart library	Government dashboard analytics visualization
Lucide React	0.468.0	Icon library	Consistent, tree-shakable icon set
next-themes	0.4.6	Theme management	Dark/light theme toggle with SSR support
clsx + tailwind-merge	Latest	Class utilities	Clean conditional styling, Tailwind class conflict resolution

Typography

- **Public Sans** (Latin) — Clean, neutral sans-serif designed for government/public sector use
- **Noto Sans Devanagari** — Google's comprehensive Devanagari script font for Hindi and related languages
- **Material Symbols Outlined** — Google's icon font for material design icons

Backend & AI Services

Service	Provider	Purpose	Cost
GPT-4o-mini	GitHub Models	Response generation for all 5 agents	Free tier
Phi-4 Mini	GitHub Models	Intent classification and agent routing	Free tier
Azure Translator	Azure Cognitive Services	Bidirectional translation for 22 languages	Pay-per-use
Azure Speech	Azure Cognitive Services	STT (Speech-to-Text) and TTS (Text-to-Speech)	Pay-per-use
Azure Computer Vision	Azure Cognitive Services	Grievance photo analysis (v3.2 API)	Pay-per-use
Azure Content Safety	Azure Cognitive Services	Text and image moderation	Pay-per-use
Azure AI Search	Azure	Government scheme search index	Pay-per-use

Azure Maps	Azure	Location and DIGIPIN features	Pay-per-use
MyMemory	Free API	Translation fallback (5000 words/day free)	Free
Fast2SMS / MSG91	Indian SMS providers	SOS SMS dispatch	Pay-per-SMS
Exotel	Indian telephony	SOS automated voice calls	Pay-per-call

DevOps

Technology	Purpose
Docker	Multi-stage build (Alpine): deps → build → runner, health checks
Netlify	Static hosting with @netlify/plugin-nextjs
Capacitor 8.x	Android native shell
ESLint 8.x	Code linting
PostCSS	CSS processing pipeline

---

7. AI/ML Pipeline — Deep Dive

7.1 Agent Routing Pipeline

The routing pipeline is the most sophisticated piece of engineering in Bharat Setu. It processes every user message through a cascading classification system:

```
User Input (any of 22 languages)
    │
    ├─ Layer 1: Raw Script Detection (sub-ms)
    │   └─ 11 regex patterns for Hindi (Devanagari)
    │   └─ Patterns for banking, health, schemes, legal, civic
    │
    ├─ Layer 2: Azure Translator (100-500ms, cached)
    │   └─ Primary: Azure Cognitive Services Translator (3s timeout)
    │   └─ Fallback: MyMemory free API (4s timeout)
    │   └─ Cache: In-memory Map (persists across requests)
    │
    ├─ Layer 3: English Keyword Overrides (sub-ms)
    │   └─ 12 regex patterns for translated text
    │   └─ "ration card" → yojana_saathi
    │   └─ "domestic violence" → vidhi_sahayak
    │   └─ "feeling sick" → swasthya_sahayak
    │
    ├─ Layer 4: Phi-4 Mini Instruct (2-5s, cached)
    │   └─ Zero-shot classification via GitHub Models
    │   └─ 5s race timeout (non-blocking)
```

```
    │   └─ Cache: In-memory Map (persists across requests)
    │
    ├─ Layer 5: Local TF-IDF/SVC (50-200ms)
    │   └─ Python classifier on localhost:5001
    │   └─ 3s timeout
    │
    ├─ Layer 6: Background GPT-4o Verification (parallel)
    │   └─ 13 few-shot examples for India-specific routing
    │   └─ 10s timeout
    │   └─ Overrides Phi-4 if it disagrees
    │
    └─ Layer 7: Client-Side Keywords (browser)
        └─ Last resort fallback
```

7.2 Response Generation Pipeline

After routing determines the correct agent:

1. **System prompt construction** — Builds agent-specific system prompt with user's language, DIGIPIN, citizen profile (Aadhaar data), and shared MCP context from previous agent interactions
2. **Conversation history** — Includes last 6 messages (3 turns) to save tokens
3. **Model selection** — Chooses between Azure OpenAI and GitHub Models based on configuration. If Azure OpenAI is configured with both endpoint and key, it's preferred; otherwise GitHub Models (free tier) is used
4. **Round-robin deployment** — When Azure OpenAI is used, requests alternate between two deployments (A/B) to double effective throughput
5. **Rate limit handling** — If deployment A returns 429, try deployment B. If both fail, fall back to GitHub Models. If all fail, return a rich demo response
6. **Post-generation Phi harvest** — After GPT finishes, check if the background Phi classification completed (100ms final check). If it suggests a different agent, include it as a handoff suggestion

7.3 Caching Strategy

Three in-memory caches persist across requests within the same server process:

- **Translator Cache** — `Map<string, string>` keyed by first 200 chars of input text
- **Routing Cache** — `Map<string, AgentKey>` keyed by normalized first 150 chars
- **Both caches avoid repeated API calls** for identical or similar queries

---

8. API Endpoints Reference

Endpoint	Method	Request Body	Response	External Service
`/api/health`	GET	—	`{ status: "ok" }`	—
`/api/agent`	POST	`{ message, agentKey, language, conversationHistory, citizenProfile, digipin }`	`{ reply, agent, suggestedAgent, resolvedAgentKey }`	Phi-4, GPT-4o-mini, Azure Translator
`/api/stt`	POST	FormData: `audio` (File), `language` (string)	`{ text }`	Azure Speech

`/api/voice`	POST	`{ text, language }`	Audio stream	Azure Speech
`/api/translate`	POST	`{ text, from?, to }`	`{ translatedText }`	Azure Translator
`/api/grievance`	POST	FormData: `description, category, digipin, language, image?`	`{ success, grievance, message }`	Azure Vision, Content Safety, Phi-4
`/api/schemes`	POST	`{ query, filters?, top? }`	`{ schemes[], total, source }`	Azure AI Search
`/api/content-safety`	POST	`{ text }`	`{ safe, categories }`	Azure Content Safety
`/api/vision-chat`	POST	FormData: `image` (File)	`{ analysis, caption, tags, objects }`	Azure Vision (v3.2)
`/api/sos`	POST	`{ userId, userName, location, context }`	`{ eventId, results[], allNotified }`	Fast2SMS/MSG91
`/api/sos/status`	GET	Query: `eventId`	`{ results[] }`	—
`/api/sos/update-location`	POST	`{ eventId, lat, lng }`	`{ updated }`	—
`/api/sos/sms`	POST	`{ phone, message }`	`{ sent }`	Fast2SMS
`/api/intelligence`	POST	`{ message }`	`{ agents[], collaboration }`	Phi-4
`/api/generate-form`	POST	`{ context }`	`{ form }`	—

---

9. State Management & Data Model

Bharat Setu uses **Zustand 5.0** for global state management. The store is defined in `src/lib/store.ts` (306 lines) and manages:

Authentication: `isAuthenticated`, `userType` (citizen/government), `role` (citizen/contributor/community_head/government)

Overlay Management: `activeOverlay` controls which full-screen overlay is displayed (agent-chat, grievance, scheme-scanner, voice, impact, sos-active, digipin, track, emergency-contacts)

Agent Chat: Per-agent chat history stored as `Record<AgentKey, ChatMessage[]>`, with `activeAgent` tracking the current specialist

Citizen Profile (CitizenProfile): Complete Aadhaar-linked profile including name (Hindi + English), DOB, gender, mobile, full address, district, state, PIN code, DIGIPIN, language preference, occupation, annual income, BPL card status, ration card type, linked schemes, eligible schemes, and emergency contacts

Tracked Items: Active cases with type (grievance/scheme/health/legal/finance), status lifecycle (Active → Under Review → In Progress → Resolved), reference IDs, ETAs, portal links, and neighbourhood count

Karma & Gamification: Karma score, redeemed rewards, and role progression

Intelligence: Collective action clusters with participant counts and status

---

10. Internationalization (i18n)

Bharat Setu supports **22 Indian languages** — all Scheduled Languages of the Indian Constitution:

Hindi, English, Bengali, Telugu, Marathi, Tamil, Gujarati, Kannada, Malayalam, Punjabi, Odia, Assamese, Urdu, Nepali, Maithili, Konkani, Manipuri (Meitei), Dogri, Santali, Bodo, Kashmiri, Sindhi, and Sanskrit.

Implementation:

- A massive **647KB translations file** (`src/lib/i18n/translations.ts`) contains all UI strings in supported languages
- A custom `useTranslation` hook provides key-based string lookup with the user's selected language
- Agent AI responses are generated natively in the user's language via system prompt instructions
- Agents naturally mix English terms for scheme names (PM-KISAN, Ayushman Bharat), portal URLs (pgportal.gov.in), and technical terms (OTP, UPI, FIR)

---

11. Security & Content Safety

Azure Content Safety: All grievance descriptions are checked against Azure Content Safety API before processing. Content with severity levels above 2 in any category (violence, hate, sexual, self-harm) is rejected.

OData Injection Prevention: The schemes API sanitizes filter values by stripping single quotes before constructing OData queries.

API Timeouts: Every external API call has explicit timeouts (3s for translation, 5s for Phi routing, 10s for webhooks, 12s for GPT, 20s for Phi ticket generation) with AbortSignal.timeout() to prevent hanging requests.

Graceful Degradation: Every external service has a fallback path — Azure Translator falls back to MyMemory, Phi-4 falls back to TF-IDF, Azure OpenAI falls back to GitHub Models, and all AI services fall back to curated demo responses.

---

12. Deployment Architecture

Docker (Production)

Three-stage Alpine-based Dockerfile:

1. **deps stage** — `node:20-alpine` , installs production dependencies only (`npm ci --omit=dev`)
2. **builder stage** — Full `npm ci` + `npm run build` for Next.js standalone output
3. **runner stage** — Minimal image with non-root user (`nextjs:nodejs`), copies standalone output + static + public, health check on `/api/health` every 30 seconds

Netlify (Static Hosting)

Configured via `netlify.toml` with `npm run build` command, `@netlify/plugin-nextjs` plugin, and Node 20 build environment.

Android (Capacitor)

`capacitor.config.ts` defines the app as "Bharat Setu" with `com.bharatsetu.app` package ID, pointing to `localhost:3000` in development.

---

13. File-Level Project Map

Components (28 files, ~420KB total)

Component	Size	Description
AgentChat.tsx	86KB	Full conversational chat interface with all 5 agents
Onboarding.tsx	78KB	Multi-step onboarding and citizen profile creation
SOSButton.tsx	46KB	SOS emergency trigger with hold-to-activate
VoiceAssistant.tsx	40KB	Voice interface with STT/TTS pipeline
ImpactDashboard.tsx	33KB	Civic karma dashboard with stats and rewards
GrievanceForm.tsx	25KB	Grievance filing with image upload
DigipinLocator.tsx	22KB	DIGIPIN encode/decode interface
RichChatCard.tsx	19KB	Structured response cards with actions
TrackCasesOverlay.tsx	19KB	Case tracking with status timeline
GovCaseManagement.tsx	18KB	Government case management interface
GovAnalytics.tsx	17KB	Government analytics dashboard
ProfileTab.tsx	15KB	User profile management
SchemeScanner.tsx	15KB	Scheme search and matching UI
GovAlerts.tsx	15KB	Government alert management
GovDashboard.tsx	15KB	Government overview dashboard
GovAdmin.tsx	14KB	Administrative controls
CivicDigitalTwin.tsx	12KB	Predictive analytics and hotspot mapping
EmergencyContactsManager.tsx	11KB	Emergency contact management
GovStatusBar.tsx	10KB	Real-time status indicators
NagarPulse.tsx	9KB	Community pulse monitoring
LoginScreen.tsx	7KB	Authentication interface
ScreenDrawer.tsx	5KB	Slide-out drawer navigation

SOSHardwareTrigger.tsx	4KB	Device button event listener
BottomNav.tsx	4KB	Bottom navigation bar
GovBottomNav.tsx	3KB	Government portal navigation
ThemeToggle.tsx	2KB	Dark/light mode toggle
VoiceFAB.tsx	2KB	Floating action button for voice
ThemeProvider.tsx	1KB	Theme context provider

Screen Components (3 files)

Component	Size	Description
BureaucracyXRay.tsx	15KB	Government machinery transparency view
CivicKarma.tsx	14KB	Karma and gamification detailed screen
SchemeScanner.tsx	15KB	Alternative scheme scan interface

Library (9 files + 2 subdirectories)

File	Size	Description
translations.ts	647KB	All UI strings in 22 languages
demo-data.ts	19KB	Demo/fallback data sets
sos-engine.ts	17KB	SOS engine with DIGIPIN + responder dispatch
store.ts	11KB	Zustand state management (306 lines)
intelligence.ts	10KB	Trust scores, collective action, predictions
web-stt.ts	8KB	Browser audio capture + WAV/OGG encoding
azure-config.ts	6KB	Centralized Azure service configuration
screens.ts	2KB	Screen definitions for drawer navigation
permissions.ts	1KB	RBAC permission system

Services (4 files)

File	Size	Description
userService.ts	2KB	User profile CRUD operations
identityService.ts	2KB	Aadhaar identity verification
grievanceService.ts	2KB	Grievance persistence layer
sosService.ts	1KB	SOS alert service

---

14. Conclusion

Bharat Setu represents a comprehensive approach to solving India's digital governance accessibility challenge. By combining:

- **Multi-model AI orchestration** (7-layer routing with 4 fallback levels)
- **22-language voice-first interaction**
- **Actionable workflows** (grievances, schemes, SOS, forms)
- **Civic gamification** (karma, roles, collective action)
- **Predictive intelligence** (digital twin, trust scores)
- **Government dashboard** (case management, analytics)

...the platform creates a unified bridge between Indian citizens and their government services, regardless of language, literacy level, or technical ability.

The codebase is well-structured with approximately **420KB of React components**, **53KB of API route logic**, **80KB of library code**, and **647KB of translations** — totaling a production-ready application that deploys as a PWA, Docker container, or native Android app.

Report generated on 23 March 2026 Project: Bharat Setu v2.0.0 Repository: Omkarpmath/Bharat-setu